

Performance Testing Report

Date	26 June 2026
Team ID	SWTID-2026-9014
Project Name	AdForge AI: AI-Powered Advertisement Copy Generator
Maximum Marks	5 Marks

Performance Testing Report:

Performance testing is critical to ensure your solution is fast, scalable, and stable under specific workloads. AdForge AI was subjected to a comprehensive performance testing process to evaluate the speed, stability, and reliability of the application under various load conditions. Since AdForge AI is a Generative AI-powered web application, performance is measured across two distinct layers — the Flask backend serving static routes and handling request validation, and the Groq API layer responsible for AI inference using the Qwen3-32B model. Four test types were conducted including Load Testing, Stress Testing, Spike Testing, and Endurance Testing using Postman, Python request scripts, and browser DevTools. All 15 Key Performance Indicators measured during testing met or exceeded their target thresholds, confirming that the application is stable, responsive, and capable of handling real-world demonstration and academic submission workloads without crashes, memory leaks, or unhandled errors.

Step 1: Test Scenarios & Tooling

S.No	Test Type	Scenario / API Endpoint Tested	Tool(s) Used
1	Load Testing	<i>Simulating 10 to 20 concurrent users simultaneously submitting campaign briefs to the /api/generate endpoint with all three platforms selected. Testing whether the Flask development server handles multiple simultaneous ad</i>	<i>Postman Collection Runner · Python requests library with threading · Browser DevTools Network tab</i>

		<i>generation requests without dropping connections or returning errors.</i>	
2	Stress Testing	<i>Pushing the /api/generate, /api/refine, and /api/tips endpoints beyond normal capacity by sending 50 rapid consecutive requests with maximum input field lengths to identify the breaking point of the Flask server and the Groq API rate limit threshold.</i>	<i>Postman Collection Runner · Python requests library · Manual rapid form submission testing</i>
3	Spike Testing	<i>Simulating a sudden burst of 15 simultaneous users accessing the application at the same time after a period of zero traffic — mimicking a real-world scenario where a professor or evaluator group accesses the application simultaneously during a demonstration or submission review.</i>	<i>Browser DevTools · Multiple browser tabs opened simultaneously · Postman parallel request execution</i>
4	Endurance Testing	<i>Running the Flask development server continuously for 2 hours with periodic requests sent every 5 minutes to the /api/generate endpoint to monitor for memory leaks, server crashes, connection timeouts, or degraded response times over an extended operational period.</i>	<i>Python automated request script with time.sleep() · Windows Task Manager for CPU and memory monitoring · Flask server console log monitoring</i>

Step 2: Key Performance Indicators (KPIs) & Results

S.No	Metric (KPI)	Target Threshold	Actual Result	Status
1	Average Response Time — Static Pages	<i>< 500 ms</i>	<i>120 ms — Homepage, About, and How It Works pages loaded within 120 ms on average over 20 test requests on localhost</i>	☒ Pass
2	Average Response Time — AI Generation	<i>< 15 seconds</i>	<i>8 to 12 seconds average for /api/generate with all three platforms selected — dependent on Groq API inference speed for Qwen3-32B at 4096 max tokens</i>	☒ Pass
3	Average Response Time — Refinement	<i>< 12 seconds</i>	<i>6 to 9 seconds average for /api/refine endpoint with standard advertisement length input and detailed feedback instructions</i>	☒ Pass
4	Average Response Time — Performance Tips	<i>< 10 seconds</i>	<i>5 to 8 seconds average for /api/tips endpoint at 2048 max tokens which is lower than the generation endpoint</i>	☒ Pass

5	Peak Response Time	<i>< 20 seconds</i>	<i>18 seconds recorded during peak load when 5 concurrent users submitted /api/generate requests simultaneously — within acceptable threshold for an AI-powered application</i>	☒ Pass
6	Throughput — Static Routes	<i>> 50 requests/sec</i>	<i>Approximately 80 to 100 requests per second achieved for GET routes (/ , /about, /how-it-works) as they require no AI processing and are served directly by Flask</i>	☒ Pass
7	Throughput — AI Routes	<i>> 1 request/sec</i>	<i>Approximately 1 to 2 requests per second achieved for AI endpoints — limited by Groq API inference time which is expected behaviour for a generative AI application</i>	☒ Pass
8	Error Rate — Validation Errors	<i>0% for valid inputs</i>	<i>0% error rate recorded for all correctly formed requests with all required fields</i>	☒ Pass

			<i>populated. Validation correctly returned 400 responses only for intentionally incomplete test inputs</i>	
9	Error Rate — Safety Filter	<i>100% block rate for harmful inputs</i>	<i>100% of test inputs containing blocked terms were correctly rejected with 422 responses before reaching the Groq API — zero harmful requests passed through the safety filter</i>	☒ Pass
10	Error Rate — Server Crashes	<i>0% during normal load</i>	<i>Zero server crashes recorded during all load, stress, spike, and endurance testing sessions. Flask development server remained stable throughout all test scenarios</i>	☒ Pass
11	CPU Utilisation	<i>< 80% under load</i>	<i>CPU utilisation remained between 15% and 35% during normal operation on a standard laptop with Intel Core i5 processor. Peaked at 45% during simultaneous 5-user stress test — well</i>	☒ Pass

			<i>within acceptable threshold</i>	
12	Memory Utilisation	<i>< 80% under load</i>	<i>Memory utilisation remained stable between 120 MB and 180 MB throughout the 2-hour endurance test. No memory leaks detected — memory usage did not increase progressively over time confirming stable stateless request handling</i>	☒ Pass
13	Groq API Rate Limit Handling	<i>Graceful error — no crash</i>	<i>When Groq API rate limit was reached during stress testing, the application correctly caught the exception in the try/except block and returned a 502 response with a user-friendly error message without crashing the server</i>	☒ Pass
14	Page Load Time — Frontend Assets	<i>< 1 second</i>	<i>CSS stylesheet and JavaScript file loaded in under 200 ms on localhost. Google Fonts loaded in under 400 ms on a</i>	☒ Pass

			<i>standard broadband connection. Total page load time under 600 ms on first visit</i>	
15	Concurrent User Handling	<i>10 concurrent users</i>	<i>Successfully handled 10 to 15 concurrent users during spike testing without connection drops, timeout errors, or degraded response quality. Each user received a complete and correctly formatted AI response</i>	☒ Pass